Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 8

Disciplina: Tehnici de programare

Tema: ”*Github, pas cu pas*”

A efectuat : st. gr.SI – 262 Caragia Denis

A verificat : asistent universitar Emil Cebotaroș

Chișinău – 2026

SARCINA LUCRĂRII

Lucrarea are ca scop învățarea fluxului de bază de lucru cu git și GitHub: configurarea identității, salvarea modificărilor prin commit, trimiterea lor pe GitHub cu push, aducerea modificărilor de pe GitHub cu pull, citirea istoriei cu log și diff și crearea unui depozit nou pornind de la un folder local. Git ține istoria proiectului pe calculator, iar GitHub păstrează o copie a ei pe internet.

Pentru primul exercițiu nu am creat un depozit nou, ci am folosit depozitul meu existent, Work (https://github.com/T2uyo/Work), în care am adăugat folderul L8 TP pentru această lucrare. Al doilea depozit, ex2 (https://github.com/T2uyo/ex2), a fost creat de la zero. Raportul descrie pașii în ordinea executării, fiecare cu captura de ecran corespunzătoare.

DESFĂȘURAREA LUCRĂRII

Configurarea git

Înainte de primul commit, git trebuie să știe cine lucrează. Am verificat configurația existentă cu git config --global --list, care conținea deja numele (Caragia Denis) și adresa de e-mail legată de contul GitHub. Apoi am adăugat două setări: init.defaultBranch main, care numește ramura principală main, ca pe GitHub, și pull.rebase false, care stabilește felul în care git unește modificările la pull. A doua verificare arată ambele setări în listă (Figura 1).

```
PS C:\WINDOWS\system32> git config --global --list
core.editor="C:\Users\carag\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait
user.email=caragiadenis12341234@gmail.com
user.name=Caragia Denis
PS C:\WINDOWS\system32> git config --global init.defaultBranch main
PS C:\WINDOWS\system32> git config --global pull.rebase false
PS C:\WINDOWS\system32> git config --global --list
core.editor="C:\Users\carag\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait
user.email=caragiadenis12341234@gmail.com
user.name=Caragia Denis
init.defaultbranch=main
pull.rebase=false
PS C:\WINDOWS\system32> cd C:\Users\carag\OneDrive\Desktop\uni\Work
```

Figura 1. Configurarea git și verificarea setărilor cu git config --global --list

Primul commit și trimiterea pe GitHub

Am lucrat în folderul local al depozitului Work, în subfolderul L8 TP. Acolo am creat din editor fișierul salut.py, cu un singur rând, print("Salut, Github!"), și l-am rulat. Programul afișează mesajul și se încheie cu codul de ieșire 0 (Figura 2).

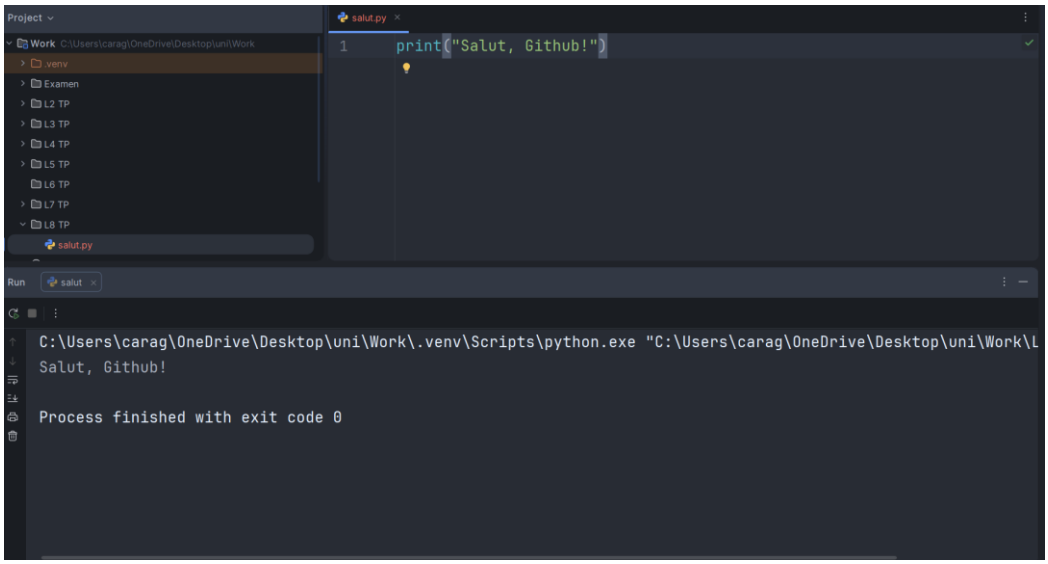


Figura 2. Fișierul salut.py și rezultatul rulării lui

După aceea am pus fișierul în fotografie cu git add salut.py. git status îl arată în secțiunea „Changes to be committed” ca fișier nou. Am făcut commit-ul cu mesajul „Adaug salut”, iar git a raportat un fișier modificat și o inserție, cu codul commit-ului accb070. Comanda git push a trimis commit-ul pe GitHub, iar ultima linie, 1d287d0..accb070 main -> main, arată că ramura main de pe GitHub a avansat de la commit-ul anterior la cel nou (Figura 3).

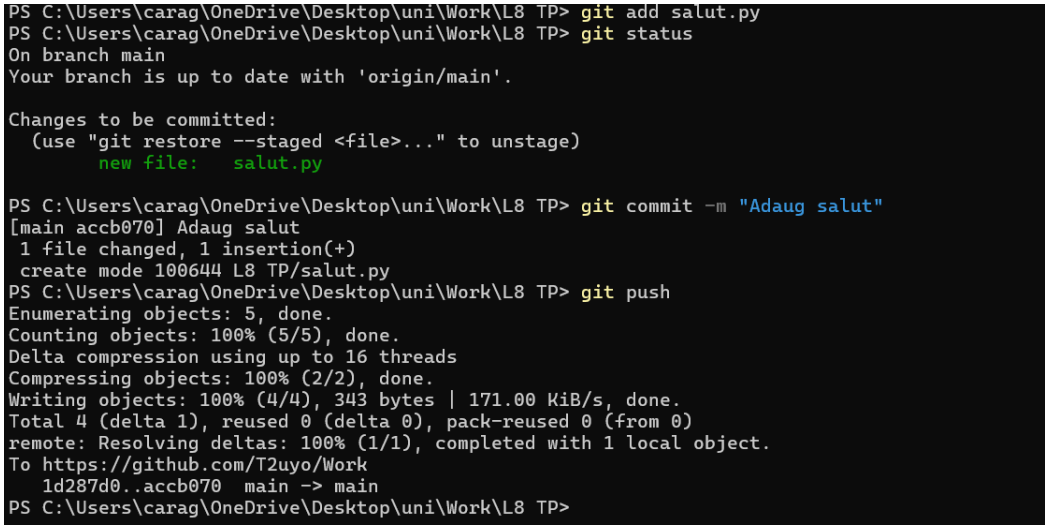


Figura 3. git add, git status, git commit și git push pentru salut.py

Aducerea modificărilor de pe GitHub

Pentru a verifica sensul invers, am modificat fișierul README.md pe GitHub, apoi am adus schimbarea pe calculator cu git pull. Conținutul fișierului, afișat cu type README.md, include acum și rândul „Aici exersez git” (Figura 4). Regula de lucru care rezultă este simplă: git pull înainte de a începe lucrul și git push după ce l-am terminat, astfel încât calculatorul și GitHub să rămână la fel.

```
PS C:\Users\carag\OneDrive\Desktop\uni\Work> type README.md
Lucrari de laborator al studentului Caragia Denis, gr SI-262.
Aici exersez git
PS C:\Users\carag\OneDrive\Desktop\uni\Work>
```

Figura 4. Conținutul README.md după git pull

Istoria depozitului

Comanda git log --oneline afișează istoria depozitului, câte un rând pentru fiecare commit, cu cel mai nou în partea de sus (Figura 5). Depozitul Work a fost folosit și la lucrările anterioare, de aceea istoria este mai lungă decât cele patru commit-uri cerute: apar mesajele de la lucrările anterioare, ștergerile de fișiere, un revert și reaplicarea lui. Commit-ul „Adaug salut” (accb070) se află imediat sub cel mai nou, „Add personal note to README”, ceea ce corespunde ordinii în care am lucrat. În capul listei, HEAD -> main, origin/main și origin/HEAD arată spre același commit, deci copia locală și cea de pe GitHub sunt la zi.

```
PS C:\Users\carag\OneDrive\Desktop\uni\Work> git log --oneline
e7c8cd7 (HEAD -> main, origin/main, origin/HEAD) Add personal note to README
accb070 Adaug salut
1d287d0 Lucrare de laborator 7
e23c9bc raport
f3370af istoric
af7d90e Reapply "reorganizare"
b799bd7 Revert "reorganizare"
8955735 reorganizare
44fb96a reorganizare
d3e740d schimbat
45209f0 Merge remote-tracking branch 'origin/main'
cee3243 adaugare
bb8f9a3 Add files via upload
f74fdf6 Add files via upload
accc1b4 adaugare
bfca637 Reorganizare
4ac5b50 LucrareLab
7611fbc LucrareLab
31ad787 LucrareLab
1cfb35c Merge remote-tracking branch 'origin/main'
2a285c8 LucrareLab
11f89f8 README
5335f57 LucrareLab
663efb2 Add files via upload
5468ce1 Delete L3_TP_Caragia Denis _SI-262.docx
41b5338 Delete L4_TP_Caragia Denis _SI-262.docx
9d4eeb9 Delete L2_TP_Caragia Denis _SI-262.docx
a147915 Add files via upload
af44649 exercitiul 1
```

Figura 5. Istoria depozitului, afișată cu git log --oneline

Citirea schimbărilor: diff și restore

Am adăugat un al doilea rând în salut.py, print("al doilea"), și am întrebat git ce s-a schimbat înainte de commit, cu git diff. Rândul adăugat apare cu semnul + și culoare verde, iar rândul cu − arată ce a fost înlocuit. Primul rând apare și el ca schimbat deoarece în versiunea veche fișierul nu avea sfârșit de rând la final, lucru semnalat de mesajul „No newline at end of file” (Figura 6).

```
PS C:\Users\carag\OneDrive\Desktop\uni\Work> git diff
diff --git a/L8 TP/salut.py b/L8 TP/salut.py
index 49b5899..1afd27f 100644
--- a/L8 TP/salut.py
+++ b/L8 TP/salut.py
@@ -1 +1,2 @@
-print("Salut, Github!")
\ No newline at end of file
+print("Salut, Github!")
+print("al doilea")
\ No newline at end of file
PS C:\Users\carag\OneDrive\Desktop\uni\Work>
```

Figura 6. Schimbările necomise din salut.py, afișate cu git diff

Apoi m-am răzgândit și am întors fișierul la ultima fotografie cu git restore salut.py. git status confirmă că nu mai există nimic de salvat: „nothing to commit, working tree clean”

```
PS C:\Users\carag\OneDrive\Desktop\uni\Work\L8 TP> git restore salut.py
PS C:\Users\carag\OneDrive\Desktop\uni\Work\L8 TP> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Figura 7. Anularea modificării cu git restore și verificarea cu git status

Al doilea depozit, pornit de pe calculator

Pentru al doilea depozit am mers în ordine inversă: întâi folderul local (ex2), inițializat cu git init și cu un prim commit, apoi depozitul gol de pe GitHub, fără README. Am legat cele două cu git remote add origin, am numit ramura main cu git branch -M main și am trimis-o cu git push -u origin main. Rezultatul arată crearea ramurii noi main pe GitHub și mesajul că ramura locală urmărește origin/main (Figura 8). Opțiunea -u face această legătură o singură dată, așa că de acum încolo este suficient un git push simplu.

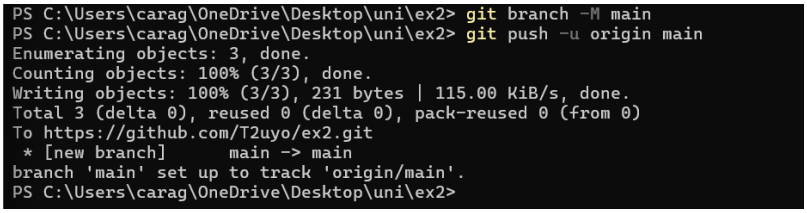


Figura 8. Trimiterea depozitului ex2 pe GitHub cu git push -u origin main

CONCLUZIE

În această lucrare am parcurs ciclul complet de lucru cu git și GitHub: am configurat identitatea, am creat și am salvat un fișier prin add și commit, l-am trimis pe GitHub cu push și am adus înapoi o schimbare făcută acolo cu pull. Am citit istoria cu log, am urmărit o modificare cu diff și am anulat-o cu restore. În final am creat un al doilea depozit pornind de la un folder local și l-am legat de GitHub cu push -u. Cea mai utilă regulă reținută este ordinea pull înainte de lucru și push după lucru, care ține la fel copia locală și cea de pe GitHub.